Inside the config repo, you can store properties directly in Spring format as *.properties or *.yml files.

Continuing with the Catalog Service example, let's define an external property for the welcome message. Navigate to the config-repo folder and create a catalog-service.yml file. Then define a value for the `polar.greeting` property used by the Catalog Service.

Listing 4.8 Defining a new message for when the config server is used

```
polar:
  greeting: "Welcome to the catalog from the config server"
```

Next, create a catalog-service-prod.yml file, and define a different value for the `polar.greeting` property to be used only when the `prod` profile is active.

Listing 4.9 Defining a new message for when the `prod` profile is active

```
polar:
  greeting: "Welcome to the production catalog from the config server"
```

Finally, commit and push your changes to the remote repository.

How does Spring Cloud Config resolve the correct configuration data for each application? How should you organize the repository to host properties for multiple applications? The library relies on three parameters to identify which property file to use to configure a specific application:

- `{application}`—The name of the application as defined by the `spring .application.name` property.
- `{profile}`—One of the active profiles defined by the `spring.profiles.active` property.